DÉPLOIEMENT D'APPLICATIONS INTELLIGENTES · MLOps

TP 7 — Automatiser les workflows MLOps

Durée : 3 h · Partie 5 du cours · Windows PowerShell

Un pipeline qu'il faut lancer à la main n'est pas automatisé. Ce TP transforme le pipeline du TP 4 en workflow planifié, surveillé depuis une interface web, capable de se déclencher tout seul, de s'arrêter sur une règle de qualité et de notifier l'équipe. Deux outils open source, tous deux présentés dans la partie 5 du cours et utilisés entièrement en local : Prefect pour l'orchestration (workflow écrit en Python, piloté depuis une interface web), et Node-RED pour l'automatisation low-code (flux construits à la souris) : un bouton qui relance le réentraînement, et une surveillance du modèle en service qui alerte quand il tombe en panne.

VERSION WINDOWS

Toutes les commandes sont écrites pour Windows PowerShell (5.1 ou 7), jamais l'invite cmd.exe. Les blocs `@' ... '@` écrivent un fichier ou l'envoient à Python : le marqueur de fermeture `'@` doit être en tout début de ligne, et le bloc se colle en entier. Les fichiers sont écrits en `-Encoding ascii` : l'encodage UTF8 de PowerShell 5.1 ajoute un marqueur invisible en tête de fichier qui casse la lecture des fichiers YAML et JSON.
Si l'activation de l'environnement est refusée : `Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned`, puis relancez `.venv\Scripts\Activate.ps1`.

CE QUI CHANGE DANS CETTE VERSION

La chaîne GitHub Actions (qui exigeait un compte GitHub et des données accessibles en ligne) est remplacée par Node-RED, l'outil d'automatisation visuelle open source de la partie 5 : tout se fait sur votre poste. Les étapes 1 à 5 (Prefect) ne changent pas. Toutes les sorties ont été obtenues réellement avec Prefect 3.9.0, Node-RED 5.0.9 et MLflow 3.17.2 ; les captures de Node-RED viennent de cette exécution.

Objectifs pédagogiques

- Transformer un script en workflow composé de tâches observables
- Piloter et surveiller les exécutions depuis une interface web
- Planifier un déclenchement périodique sans intervention humaine
- Interrompre automatiquement le workflow sur une règle de qualité
- Construire sans code un flux qui déclenche le workflow et surveille le modèle en service
- Versionner un flux low-code comme du code

Prérequis

- TP 4 terminé : le pipeline DVC fonctionne (`dvc repro`)
- TP 6 terminé : le modèle `fraude-flaml` est dans le registre MLflow avec l'alias `production`
- Node.js 18 ou plus récent (nodejs.org, version LTS) pour Node-RED

Installation

Prefect fournit un serveur local avec interface web, sans base de données à installer. Node-RED s'installe avec npm, le gestionnaire de paquets de Node.js.

terminal

```
cd tp-mlops; .venv\Scripts\Activate.ps1
pip install prefect
prefect version
node --version                          # 18 ou plus
npm install -g node-red
```

RÉSULTAT ATTENDU

```
Version:             3.9.0         (ou plus recent)
...
v22.x.x
added 276 packages in 26s
```

Déroulement

Étape 1 — Démarrer le serveur d'orchestration

Laissez le terminal 1 ouvert : l'interface web s'y appuie. Dans le terminal 2, indiquez une fois pour toutes à Prefect l'adresse de ce serveur ; sans ce réglage, chaque script démarre son propre serveur temporaire et rien n'apparaît dans l'interface.

terminal 1

```
prefect server start
```

RÉSULTAT ATTENDU

```
Check out the dashboard at http://127.0.0.1:4200

Ouvrez cette adresse : le tableau de bord est vide pour l'instant.
```

terminal 2

```
prefect config set PREFECT_API_URL=http://127.0.0.1:4200/api   # une seule fois
```

RÉSULTAT ATTENDU

```
Set 'PREFECT_API_URL' to 'http://127.0.0.1:4200/api'.
Updated profile 'ephemeral'.
```

Si ça coince : si le port 4200 est occupé : `prefect server start --port 4300`, puis adaptez `PREFECT_API_URL` en conséquence.

Étape 2 — Écrire le workflow

Chaque étape devient une tâche observable, avec ses réessais. Le workflow appelle le pipeline DVC existant : on n'a rien réécrit. Vérifiez d'abord que les paramètres sont bien ceux de référence.

terminal 2

```
Select-String -Path params.yaml -Pattern 'n_estimators|max_depth'
```

RÉSULTAT ATTENDU

```
  n_estimators: 300
  max_depth: 12

Si vous lisez 600 et 20, ce sont les reglages du dernier dvc exp run du TP 4 :
remettez 300 et 12 dans params.yaml avant de continuer.
```

flow_mlops.py

```
@'
import json, subprocess, yaml
from prefect import flow, task, get_run_logger
@task(retries=2, retry_delay_seconds=10)          # 2 reessais en cas d'echec
def rejouer_pipeline():
    r = subprocess.run(['dvc', 'repro'], capture_output=True, text=True)
    get_run_logger().info(r.stdout[-800:])
    if r.returncode != 0:
        raise RuntimeError(r.stderr[-500:])
    return 'pipeline rejoue'
@task
def lire_metriques():
    return json.load(open('reports/metrics.json'))
@task
```

```
def valider(mets):
    seuil = yaml.safe_load(open('params.yaml'))['validation']['seuil_f1']
    if mets['f1'] < seuil:                        # regle de qualite
        raise ValueError(f"F1 {mets['f1']} < seuil {seuil} : modele refuse")
    get_run_logger().info(f"F1 {mets['f1']} >= {seuil} : modele accepte")
    return True
@task
def notifier(mets):
    get_run_logger().info(f"NOTIFICATION equipe : nouveau modele F1={mets['f1']}")
@flow(name='reentrainement-fraude', log_prints=True)
def reentrainement():
    rejouer_pipeline()
    mets = lire_metriques()
    valider(mets)
    notifier(mets)
if __name__ == '__main__':
    reentrainement()
'@ | Set-Content -Encoding ascii flow_mlops.py
python flow_mlops.py
```

RÉSULTAT ATTENDU

```
Beginning flow run 'xxx' for flow 'reentrainement-fraude'
... F1 0.8636 >= 0.82 : modele accepte
... NOTIFICATION equipe : nouveau modele F1=0.8636
Finished in state Completed()

Dans l'interface, l'execution apparait avec ses quatre taches et leurs journaux.
```

Étape 3 — Vérifier l'arrêt automatique sur la règle de qualité

On dégrade volontairement le modèle : le workflow doit s'arrêter et signaler l'échec.

terminal 2

```
(Get-Content params.yaml) -replace 'max_depth: 12', 'max_depth: 2' |
    Set-Content -Encoding ascii params.yaml                    # modele trop simple
python flow_mlops.py                                           # doit echouer
(Get-Content params.yaml) -replace 'max_depth: 2', 'max_depth: 12' |
    Set-Content -Encoding ascii params.yaml                    # on remet la valeur
```

RÉSULTAT ATTENDU

```
Task run 'valider-...' - Finished in state Failed('Task run encountered an
exception ValueError: F1 0.2013 < seuil 0.82 : modele refuse')
Flow run '...' - Finished in state Failed(...)

Dans l'interface, l'execution est en rouge et la tache fautive est identifiee.
Aucun modele n'a ete promu.
```

Relancez `python flow_mlops.py` : DVC retrouve dans son cache le modèle de profondeur 12 (« Stage 'entrainer' is cached ») sans le réentraîner, et le workflow repasse au vert avec F1 = 0,8636.

Étape 4 — Planifier le déclenchement automatique

Le workflow doit s'exécuter chaque lundi à 6 h sans intervention. Le worker est le processus qui exécute les workflows planifiés.

terminal 2

```
prefect work-pool create --type process pool-tp         # file d'execution
prefect --no-prompt deploy flow_mlops.py:reentrainement `
  --name hebdomadaire --pool pool-tp --cron '0 6 * * 1'
```

terminal 3 : le worker qui exécute

```
cd tp-mlops; .venv\Scripts\Activate.ps1
prefect worker start --pool pool-tp
```

RÉSULTAT ATTENDU

```
Deployment 'reentrainement-fraude/hebdomadaire' successfully created with id
'3dfd0fc8-b14f-4f8d-8809-5fee2842b70d'.
Worker 'ProcessWorker xxx' started!

Onglet Deployments de l'interface : la prochaine execution planifiee s'affiche.
```

Si ça coince : l'option `--no-prompt` évite les questions interactives : Prefect indique qu'il ne trouve pas de `prefect.yaml` et que le worker chargera le flux depuis le dossier local, c'est normal. Notez l'identifiant (id) du déploiement affiché : il servira à l'étape 7.

Étape 5 — Déclencher une exécution depuis l'interface

Sans taper de commande : c'est le mode d'utilisation d'un profil métier. Dans l'interface : Deployments, ouvrir hebdomadaire, bouton Run puis Quick run. Suivre l'exécution en direct, ouvrir les journaux de chaque tâche, puis retrouver l'exécution dans Runs avec sa durée et son état.

RÉSULTAT ATTENDU

```
L'execution apparait immediatement, passe par Running puis Completed.
Le terminal 3 (worker) affiche son deroulement.
```

Étape 6 — Installer et démarrer Node-RED (10 min)

Node-RED est un outil d'automatisation visuelle open source (licence Apache 2.0, fondation OpenJS). On y construit des **flux** en reliant des **nœuds** : un nœud reçoit un message (`msg`), le transforme, et le passe au suivant. Le contenu utile du message est dans `msg.payload`. Rien ne s'exécute tant que le flux n'est pas **déployé** (bouton rouge **Deploy** en haut à droite).

terminal 4 : Node-RED

```
cd tp-mlops
node-red
```

RÉSULTAT ATTENDU

```
Welcome to Node-RED
...
[info] Server now running at http://127.0.0.1:1880/
[info] Started flows
```

1. Ouvrez http://127.0.0.1:1880 dans le navigateur. Fermez la visite guidée et répondez **No, do not enable notifications** à la question sur les mises à jour.
2. Repérez les zones : à gauche la **palette** (les nœuds rangés par catégorie : common, function, network, storage...), au centre l'espace de travail avec ses onglets de flux, à droite la barre latérale. Ouvrez l'onglet **Debug** de la barre latérale (icône d'insecte) : c'est là que s'afficheront les messages.
3. Renommez l'onglet `Flow 1` (double-clic sur l'onglet) en `Réentraîner d'un clic`.

Si ça coince : si PowerShell refuse d'exécuter `node-red` (script non signé), lancez `node-red.cmd`, ou autorisez les scripts pour la session avec la commande `Set-ExecutionPolicy` donnée en tête de fiche.

Étape 7 — Flux 1 : relancer le réentraînement d'un clic (20 min)

Un profil métier veut relancer le réentraînement sans terminal ni interface Prefect. Node-RED va appeler l'API de Prefect pour créer une exécution du déploiement `hebdomadaire` de l'étape 4 ; le worker (terminal 3) l'exécutera. Il faut d'abord l'identifiant du déploiement.

terminal 2

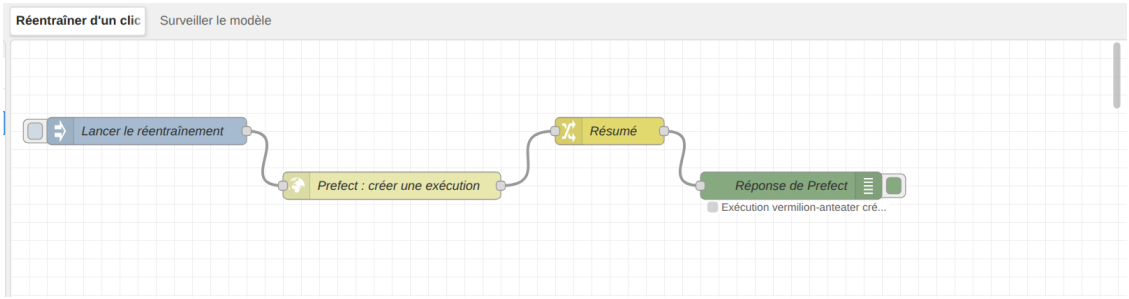
```
prefect deployment inspect 'reentrainement-fraude/hebdomadaire'
```

RÉSULTAT ATTENDU

```
{
    'id': '3dfd0fc8-b14f-4f8d-8809-5fee2842b70d',
    ...
    'name': 'hebdomadaire',

Copiez VOTRE identifiant (il figure aussi dans l'adresse de la page du deploiement
dans l'interface Prefect).
```

1. Catégorie **common** : glissez un nœud **inject**. Double-cliquez dessus : nommez-le `Lancer le réentraînement`, réglez **msg.payload** sur le type **{} JSON** avec la valeur `{}` (un objet vide). Cliquez sur **Done**.
2. Catégorie **network** : glissez un nœud **http request** et reliez la sortie de l'inject (petit carré à droite) à son entrée. Réglez **Method** = `POST`, **URL** = `http://127.0.0.1:4200/api/deployments/VOTRE_ID/create_flow_run`, **Return** = **a parsed JSON object**. Nommez-le `Prefect : créer une exécution`.
3. Catégorie **function** : glissez un nœud **change** et reliez-le. Règle : **Set** `msg.payload` **to the value**, type **J: expression** (JSONata), valeur : `'Exécution ' & payload.name & ' créée, état : ' & payload.state.name`. Nommez-le `Résumé`.
4. Catégorie **common** : glissez un nœud **debug**, reliez-le, et cochez l'option qui affiche aussi le message sous le nœud (**node status**). Nommez-le `Réponse de Prefect`.
5. Cliquez sur **Deploy**. Puis cliquez sur le petit bouton carré à gauche du nœud inject : le message part.
6. Vérifiez dans l'interface Prefect (**Runs**) et dans le terminal 3 (worker) que l'exécution a bien tourné.



Capture réelle : le flux 1 dans Node-RED 5.0.9, après un clic sur le bouton.

RÉSULTAT ATTENDU

```
Barre Debug : "Exécution vermilion-anteater créée, état : Scheduled"
Terminal 3 (worker), quelques secondes plus tard :
  Task run 'valider-...' - F1 0.8636 >= 0.82 : modele accepte
  Task run 'notifier-...' - NOTIFICATION equipe : nouveau modele F1=0.8636
  Flow run 'vermilion-anteater' - Finished in state Completed()
```

Si ça coince : le nom de l'exécution (`vermilion-anteater`) est tiré au hasard par Prefect. Si la barre Debug affiche une erreur 404, l'identifiant du déploiement est faux ; si elle affiche « ECONNREFUSED », le serveur Prefect (terminal 1) est arrêté. Une exécution créée mais qui reste « Scheduled » signifie que le worker n'est pas démarré.

Étape 8 — Flux 2 : surveiller le modèle en service (25 min)

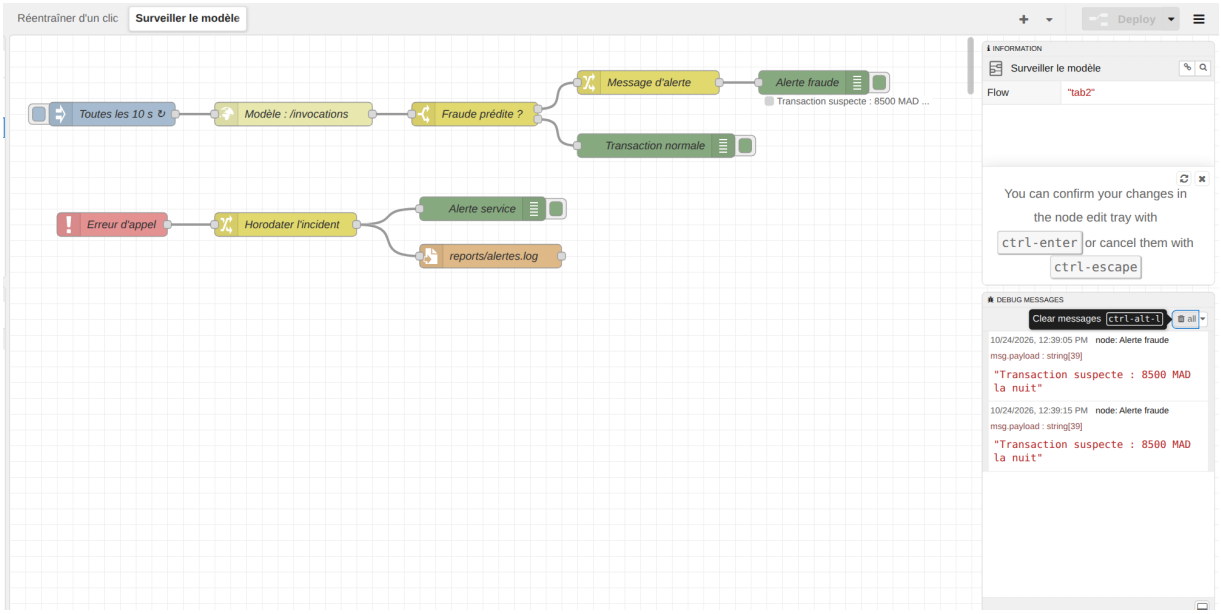
Toutes les 10 secondes, Node-RED envoie une transaction au modèle FLAML du TP 6, affiche une alerte si le modèle prédit une fraude, et surtout détecte la panne du service lui-même : il écrit alors un incident horodaté dans `reports/alertes.log`. Créez un nouvel onglet (bouton **+** au-dessus de l'espace de travail) nommé `Surveiller le modèle`. Démarrez d'abord le point de terminaison :

terminal 5 : le modèle en service

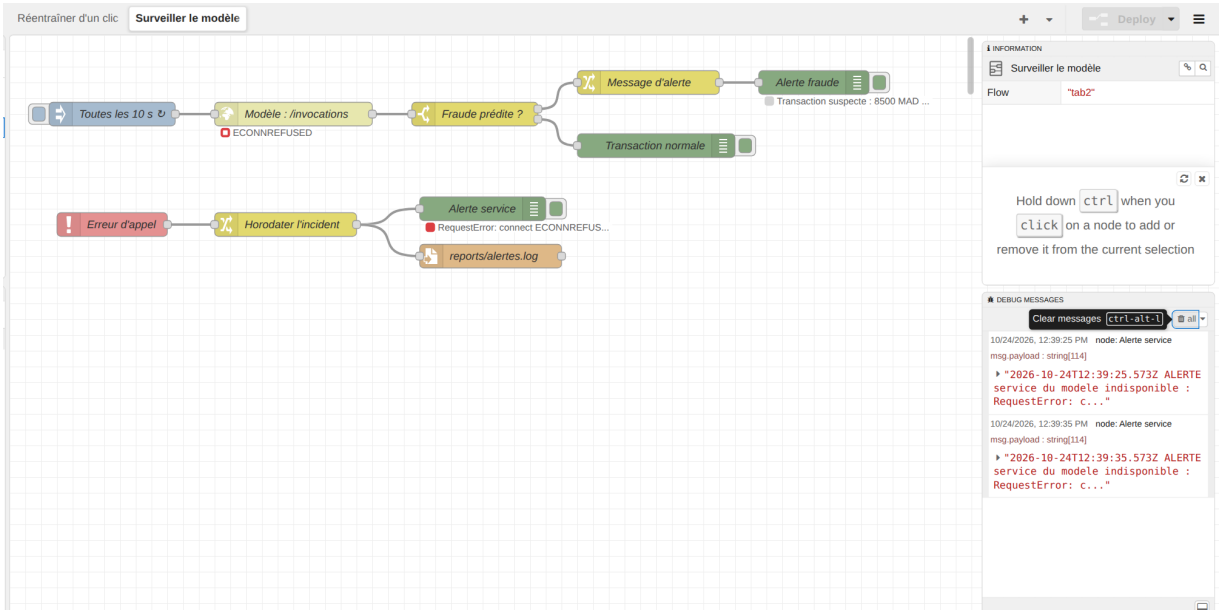
```
cd tp-mlops; .venv\Scripts\Activate.ps1
$env:MLFLOW_TRACKING_URI = "http://127.0.0.1:5000"
mlflow models serve -m 'models:/fraude-flaml@production' -p 8101 --env-manager local
```

1. **inject** nommé `Toutes les 10 s` : **msg.payload** de type **{} JSON** avec la valeur `{"dataframe_records":[{"montant":8500.0,"ancienne":3,"freq_24h":9,"nuit":1}]}` ; en bas de la fenêtre, **Repeat** = **interval**, every **10 seconds**.
2. **http request** nommé `Modèle : /invocations` : **POST**, URL `http://127.0.0.1:8101/invocations`, **Return** = **a parsed JSON object**.

3. **switch** (catégorie function) nommé `Fraude prédite ?` : **Property** = `msg.payload.predictions[0]` ; règle 1 : **==** `1` de type **number** ; ajoutez (bouton **+ add**) la règle 2 : **==** `0` de type number. Le nœud a maintenant deux sorties.
4. Sortie 1 : un **change** `Message d'alerte` qui met dans `msg.payload` le texte `'Transaction suspecte : 8500 MAD la nuit'` (type J: expression ou chaîne), puis un **debug** `Alerte fraude`. Sortie 2 : un **debug** `Transaction normale`.
5. Gestion de la panne : un nœud **catch** (catégorie common) nommé `Erreur d'appel`, réglé sur **Catch errors from : selected nodes**, en sélectionnant seulement le nœud http request. Il n'a pas d'entrée : il reçoit les erreurs de ce nœud.
6. Après le catch : un **change** `Horodater l'incident`, règle Set `msg.payload` en **J: expression** : `$now() & ' ALERTE service du modele indisponible : ' & error.message`.
7. Reliez ce change à deux nœuds : un **debug** `Alerte service`, et un nœud **write file** (catégorie storage) : **Filename** = le chemin complet de `tp-mlops/reports/alertes.log`, **Action** = **append to file**, cochez **Add newline** et **Create directory if it doesn't exist**.
8. **Deploy**. Observez la barre Debug pendant 30 secondes.
9. Simulez la panne : **Ctrl+C** dans le terminal 5. Attendez 30 secondes, lisez la barre Debug et le fichier `reports/alertes.log`. Relancez le service : les alertes de panne cessent.



Capture réelle : le flux 2 en fonctionnement normal (une fraude prédite toutes les 10 s).



Capture réelle : le même flux après l'arrêt du service du modèle.

RÉSULTAT ATTENDU

```
Service en marche, toutes les 10 s : "Transaction suspecte : 8500 MAD la nuit"
Service arrete, toutes les 10 s, dans la barre Debug et dans reports/alertes.log :
2026-10-24T12:39:25.573Z ALERTE service du modele indisponible : RequestError:
  connect ECONNREFUSED 127.0.0.1:8101
2026-10-24T12:39:35.573Z ALERTE service du modele indisponible : ...
```

POURQUOI LA RÈGLE 2 DU SWITCH EST « == 0 » ET NON « otherwise »

Lors de la préparation, le switch avait une règle « otherwise ». Quand le service est tombé, le nœud http request a aussi transmis un message contenant le texte de l'erreur : il est parti dans « otherwise » et s'est affiché comme... « Transaction normale ». Une surveillance qui classe une panne en cas normal est pire que pas de surveillance : testez toujours le cas de panne.

Étape 9 — Versionner le flux low-code et conclure (15 min)

Un flux construit à la souris n'existe que dans Node-RED tant qu'il n'est pas exporté. On l'exporte en JSON et on le commite à côté du code, comme le workflow Orange au TP 5.

1. Menu principal (les trois traits en haut à droite) > **Export** > **all flows** > **Download** : enregistrez le fichier dans `tp-mlops/nodered/flows.json`.
2. Ouvrez ce fichier dans un éditeur de texte : chaque nœud est un objet JSON avec son type, son nom, ses réglages et ses liens (`wires`). Retrouvez l'URL de Prefect et le seuil du switch.

terminal 2

```
git add flow_mlops.py nodered/flows.json
git commit -m 'auto: workflow Prefect et flux Node-RED'
```

Rédigez enfin dans `reports/tp7_synthese.md` un tableau comparant Prefect et Node-RED sur : ce qui déclenche l'exécution, qui peut la modifier, ce qui se passe en cas d'échec, comment on la versionne, et le type de tâche pour lequel chacun est adapté. Ajoutez une ligne pour une chaîne d'intégration continue (GitHub Actions, GitLab CI), vue en cours sans être pratiquée ici.

RÉSULTAT ATTENDU

```
Conclusion attendue : Prefect execute et surveille des TRAITEMENTS recurrents
ecrits en Python ; Node-RED branche ces traitements sur des evenements (un bouton,
une minuterie, une erreur) sans code ; une chaine CI valide le CODE a chaque
modification. Les trois sont complementaires.
```

Questions du compte rendu

1. Quelle différence de rôle entre un orchestrateur (Prefect), un outil d'automatisation low-code (Node-RED) et une chaîne d'intégration continue ?
2. Le workflow échoue à 6 h du matin un lundi. Comment l'équipe l'apprend-elle, et que doit-elle trouver ? Comment Node-RED pourrait-il aider ?
3. Pourquoi avoir mis deux réessais sur la tâche de pipeline, mais aucun sur la tâche de validation ?
4. Le bouton du flux 1 permet à n'importe qui de relancer le réentraînement. Quels risques voyez-vous, et quelles protections ajouteriez-vous ?
5. Expliquez l'incident de la règle « otherwise » (étape 8). Quelle leçon en tirer pour toute surveillance automatique ?
6. Quels avantages et quels risques voyez-vous à configurer un flux dans une interface graphique plutôt que dans un fichier versionné ? L'export JSON règle-t-il tous les problèmes ?
7. Quel événement, autre qu'une date ou un clic, pourrait déclencher le réentraînement ? Comment le détecteriez-vous ?

Livrables

- Le fichier `flow_mlops.py` et l'export `nodered/flows.json`, commités dans Git
- Capture du tableau de bord Prefect avec une exécution réussie et une exécution échouée

- Capture de l'onglet Deployments montrant la planification
- Captures de Node-RED : le flux 1 après un clic, le flux 2 avant et pendant la panne
- Le fichier `reports/alertes.log` produit pendant la panne
- La synthèse comparative `reports/tp7_synthese.md`

Barème

Critère	Points
Workflow Prefect découpé en tâches, exécuté et observable	4
Arrêt automatique sur la règle de qualité démontré	3
Planification et worker fonctionnels	3
Flux Node-RED 1 : réentraînement déclenché d'un clic	3
Flux Node-RED 2 : surveillance, alerte et journal de panne	4
Flux versionné, synthèse comparative et réponses aux questions	3
Total	20